



GUIA DEFINITIVO

A BÍBLIA DO BEAVERSEC

Comandos, Scripts e Configurações

B.E.A.V.E.R. – Binary Enumeration &
Advanced Vulnerability Exploitation Recon

v1.0.0 python 3.8+ MIT license

Versão 1.0.0 · Autor BeaverSec Team

Última atualização 20/06/2026

github.com/ojhonatanls/beaversec

Índice

01	Introdução
02	Estrutura do Projeto
03	Pré-requisitos
04	Formas de Execução
05	Comandos Básicos
06	Módulos Disponíveis
07	Erros Comuns e Soluções
08	Scripts Prontos para Uso
09	Comandos Rápidos (Cheat Sheet)
10	Exemplos Práticos
11	Perguntas Frequentes
12	Contribuindo

01 Introdução

BeaverSec (B.E.A.V.E.R. — *Binary Enumeration & Advanced Vulnerability Exploitation Recon*) é um canivete suíço modular para cibersegurança, construído em Python com foco em aprendizado prático e evolução contínua.

Características Principais

- ✓ **Modular:** cada ferramenta é um módulo independente
- ✓ **Leve:** usa apenas bibliotecas padrão do Python
- ✓ **Multiplataforma:** funciona no Linux, Windows e macOS
- ✓ **Extensível:** fácil adicionar novos módulos
- ✓ **Educacional:** perfeito para aprendizado de cybersecurity

02 Estrutura do Projeto

```
~/BEAVERSEC/
├── .github/ # Templates do GitHub
│   ├── ISSUE_TEMPLATE.md
│   └── PULL_REQUEST_TEMPLATE.md
├── beaversec/ # Pacote principal
│   ├── core/ # Núcleo do sistema
│   │   ├── __init__.py
│   │   └── handler.py # Gerenciador de módulos
│   ├── modules/ # Módulos (ferramentas)
│   │   ├── __init__.py
│   │   ├── ping_sweep.py # Varredura ICMP
│   │   ├── port_scanner.py # Varredura de portas
│   │   └── meu_modulo.py # Módulo de exemplo
│   ├── utils/ # Utilitários
│   │   ├── __init__.py
│   │   └── logger.py # Sistema de logging
│   └── __init__.py
├── tests/ # Testes unitários
│   └── __init__.py
├── .gitignore # Ignora arquivos desnecessários
├── BIBLIA_BEAVERSEC.md # Guia completo (este arquivo)
├── README.md # Documentação principal
├── main.py # Ponto de entrada
├── requirements.txt # Dependências
├── auto_scan.py # Varredura automática
├── install.sh # Instalação de dependências
├── scan.sh # Varredura rápida
├── setup.sh # Configuração rápida
└── update.sh # Atualização do GitHub
```

03 Pré-requisitos

Obrigatórios

- **Python 3.8+** — download em python.org
- **Git** (para clonar o repositório)

Verificando Instalação

```
# Verifica Python
python3 --version
# Saída esperada: Python 3.8.x ou superior

# Verifica Git
git --version
# Saída esperada: git version 2.x.x
```

Sistema Operacional

- ✓ Linux (Ubuntu, Debian, Fedora, etc.)
- ✓ Windows (10, 11 com WSL ou PowerShell)
- ✓ macOS (10.15+)

04 Formas de Execução

4.1 Usando python3 (Recomendado para iniciantes)

```
python3 main.py <MÓDULO> <ALVO> [OPÇÕES]
```

Exemplos:

```
python3 main.py -l
python3 main.py ping_sweep 8.8.8.8
python3 main.py ping_sweep 192.168.1.0/24 -v
```

4.2 Criando Alias (Para usar só python)

```
# Alias temporário (válido apenas na sessão atual)
alias python='python3'

# Agora pode usar:
python main.py -l
```

Para tornar permanente:

```
echo "alias python='python3'" >> ~/.bashrc
source ~/.bashrc
```

4.3 Tornando Executável (Modo Direto)

```
# Dá permissão de execução
chmod +x main.py

# Executa diretamente
./main.py -l
./main.py ping_sweep 8.8.8.8
```

4.4 Instalação Global (Modo Profissional)

```
# Cria link simbólico no sistema
sudo ln -s $(pwd)/main.py /usr/local/bin/beaver

# Agora pode executar de QUALQUER lugar!
beaver -l
beaver ping_sweep 8.8.8.8
```

Desinstalar:

```
sudo rm /usr/local/bin/beaver
```

05 Comandos Básicos

Comandos Gerais

Comando	Descrição	Exemplo
-l, --list	Lista todos os módulos disponíveis	python3 main.py -l
-h, --help	Mostra ajuda completa	python3 main.py -h
--version	Mostra a versão do BeaverSec	python3 main.py --version
-v, --verbose	Ativa modo detalhado	python3 main.py ping_sweep 8.8.8.8 -v

Estrutura do Comando

```
python3 main.py [MÓDULO] [ALVO] [OPÇÕES]
                |           |           |
                |           |           └ -v (opcional)
                |           └ IP ou CIDR
                └ Nome do módulo
```

06 Módulos Disponíveis

6.1 ping_sweep — Varredura ICMP

Descrição: descobre hosts ativos usando ping (ICMP Echo Request).

Sintaxe:

```
python3 main.py ping_sweep <ALVO> [OPÇÕES]
```

Tipos de Alvo:

Tipo	Formato	Exemplo
IP Único	X.X.X.X	8.8.8.8
Rede CIDR	X.X.X.X/Y	192.168.1.0/24

Opções:

- `-V, --verbose`: mostra cada tentativa de ping

Exemplos:

```
# Ping em IP único
python3 main.py ping_sweep 8.8.8.8

# Ping em rede inteira
python3 main.py ping_sweep 192.168.1.0/24

# Modo detalhado
python3 main.py ping_sweep 192.168.1.0/24 -v
```

Saída Esperada:

```
LISTA DE HOSTS ATIVOS:
1. 192.168.1.1
2. 192.168.1.10
3. 192.168.1.11
```

6.2 port_scanner — Varredura de Portas

Descrição: escaneia portas TCP comuns em um alvo.

Sintaxe:

```
python3 main.py port_scanner <ALVO> [OPÇÕES]
```

Exemplo:

```
python3 main.py port_scanner scanme.nmap.org -v
```

6.3 meu_modulo — Módulo de Exemplo

Descrição: template para criação de novos módulos.

Sintaxe:

```
python3 main.py meu_modulo <ALVO>
```

Exemplo:

```
python3 main.py meu_modulo 8.8.8.8
```


6.4 Criando Seu Próprio Módulo

1 Crie um arquivo em `beaversec/modules/`

```
# beaversec/modules/meu_modulo.py
def run(target: str, verbose: bool = False) -> None:
    """Função obrigatória para o módulo"""
    print(f"Executando meu módulo contra {target}")
    if verbose:
        print("Modo detalhado ativado")
```

2 Execute seu módulo

```
python3 main.py meu_modulo 8.8.8.8 -v
```

07 Erros Comuns e Soluções

Erro 1: Comando 'python' não encontrado

✗ ERRO

Comando 'python' não encontrado, você quis dizer:
comando 'python3' do deb python3

✓ SOLUÇÃO

Use `python3` no lugar de `python`

```
python3 main.py -l # OK - CORRETO
```

Erro 2: the following arguments are required: module, target

✗ ERRO

beaver: error: the following arguments are required: module, target

✓ SOLUÇÃO

O comando precisa de módulo e alvo

```
# X - ERRADO
python3 main.py -l ping_sweep

# OK - CORRETO
python3 main.py ping_sweep 8.8.8.8
```

Erro 3: Permission denied

✗ ERRO

```
./main.py -l
bash: ./main.py: Permission denied
```

✓ SOLUÇÃO

Dê permissão de execução

```
chmod +x main.py
./main.py -l # OK - FUNCIONA
```

Erro 4: ModuleNotFoundError: No module named 'beaversec'

✗ ERRO

ModuleNotFoundError: No module named 'beaversec'

✓ SOLUÇÃO

Execute da raiz do projeto

```
cd ~/BEAVERSEC
python3 main.py -l # OK - CORRETO
```

Erro 5: NENHUM HOST ATIVO ENCONTRADO

✗ ERRO

NENHUM HOST ATIVO ENCONTRADO.

Causas e Soluções:

- **Alvo incorreto:** verifique o IP ou CIDR
- **Sem conexão:** teste com `ping 8.8.8.8`
- **Firewall:** pode estar bloqueando ICMP
- **Fora da rede:** verifique se está na mesma rede

Erro 6: CIDR inválido

✗ ERRO

ValueError: CIDR inválido

✓ SOLUÇÃO

Use o formato correto

```
# X - ERRADO
python3 main.py ping_sweep 192.168.1/24

# OK - CORRETO
python3 main.py ping_sweep 192.168.1.0/24
```

08 Scripts Prontos para Uso

setup.sh

8.1 Configuração Rápida

```
#!/bin/bash
echo "Configurando BeaverSec..."

chmod +x main.py
echo "[OK] main.py agora é executável"

if ! grep -q "alias beaver=" ~/.bashrc; then
    echo "alias beaver='python3 $(pwd)/main.py'" >> ~/.bashrc
    echo "[OK] Alias 'beaver' adicionado ao .bashrc"
fi

read -p "[?] Deseja instalar globalmente? (s/N) " -n 1 -r
echo
if [[ $REPLY =~ ^[Ss]$ ]]; then
    sudo ln -sf $(pwd)/main.py /usr/local/bin/beaver
    echo "[OK] BeaverSec instalado globalmente!"
fi

echo ""
echo "Configuração concluída!"
echo "Agora você pode usar:"
echo "    beaver -l"
echo "    beaver ping_sweep 8.8.8.8"
```

Como usar:

```
chmod +x setup.sh
./setup.sh
```

scan.sh

8.2 Varredura Rápida

```
#!/bin/bash
echo "BeaverSec - Scanner Rápido"

if [ $# -eq 0 ]; then
    echo "Uso: ./scan.sh <IP>"
    echo "    ./scan.sh 8.8.8.8"
    echo "    ./scan.sh 192.168.1.0/24 -v"
    exit 1
fi

python3 main.py ping_sweep "$@"
```

Como usar:

```
chmod +x scan.sh
./scan.sh 8.8.8.8
./scan.sh 192.168.1.0/24 -v
```

auto_scan.py

8.3 Varredura Automática

```
#!/usr/bin/env python3
import subprocess
import time

targets = [
    "8.8.8.8",
    "1.1.1.1",
    "192.168.1.0/24",
]

for target in targets:
    print(f"\nEscaneando: {target}")
    subprocess.run(["python3", "main.py", "ping_sweep", target])
    time.sleep(1)

print("\n[OK] Todas as varreduras concluídas!")
```

Como usar:

```
chmod +x auto_scan.py
python3 auto_scan.py
```

```
install.sh
```

8.4 Instalação de Dependências

```
#!/bin/bash
echo "BeaverSec - Instalando dependências..."

if ! command -v python3 &> /dev/null; then
    echo "[ERR0] Python 3 não encontrado!"
    exit 1
fi

echo "[OK] Python 3 encontrado: $(python3 --version)"

if [ -f "requirements.txt" ]; then
    pip3 install -r requirements.txt
fi

chmod +x main.py
echo "[OK] Instalação concluída!"
```

```
update.sh
```

8.5 Atualização do GitHub

```
#!/bin/bash
echo "Atualizando BeaverSec..."

git pull origin main
chmod +x main.py

echo "[OK] BeaverSec atualizado!"
```

09 Comandos Rápidos (Cheat Sheet)

Comandos com python3 (Sempre funciona)

```
# Listar módulos
python3 main.py -l

# Ping em IP único
python3 main.py ping_sweep 8.8.8.8

# Ping em rede
python3 main.py ping_sweep 192.168.1.0/24

# Ping com detalhes
python3 main.py ping_sweep 8.8.8.8 -v

# Varredura detalhada de rede
python3 main.py ping_sweep 192.168.1.0/24 -v

# Ajuda
python3 main.py -h

# Versão
python3 main.py --version
```

Comandos com ./main.py (Após chmod +x main.py)

```
./main.py -l
./main.py ping_sweep 8.8.8.8
./main.py ping_sweep 192.168.1.0/24 -v
```

Comandos com beaver (Após instalação global)

```
beaver -l
beaver ping_sweep 8.8.8.8
beaver ping_sweep 192.168.1.0/24 -v
```

10 Exemplos Práticos

Exemplo 1: Descobrindo Hosts na Rede Local

```
python3 main.py ping_sweep 192.168.1.0/24 -v
```

Saída:

```
✓ 192.168.1.1 está ATIVO
✓ 192.168.1.10 está ATIVO
✓ 192.168.1.11 está ATIVO
```

LISTA DE HOSTS ATIVOS:

1. 192.168.1.1
2. 192.168.1.10
3. 192.168.1.11

Exemplo 2: Verificando se um Servidor Está Online

```
python3 main.py ping_sweep 8.8.8.8 -v
```

Exemplo 3: Usando no Windows (PowerShell)

```
python main.py -l
python main.py ping_sweep 8.8.8.8
```

Exemplo 4: Criando um Alias Permanente

Linux/macOS:

```
echo "alias beaver='python3 /home/usuario/BEAVERSEC/main.py'" >> ~/.bashrc
source ~/.bashrc
beaver -l
```

11 Perguntas Frequentes

Q1: Por que o comando python não funciona?

No Linux/macOS, o comando padrão é `python3`. Use `python3` ou crie um alias.

Q2: Como instalar o BeaverSec?

Não precisa instalar! Basta clonar o repositório e executar.

Q3: Posso usar no Windows?

Sim! Use no PowerShell com `python` ou no WSL com `python3`.

Q4: Como adicionar um novo módulo?

Crie um arquivo em `beaversec/modules/` com uma função `run()`.

Q5: Por que não encontro hosts ativos?

Verifique se está na mesma rede, se o firewall não bloqueia ICMP e se o alvo está correto.

Q6: O BeaverSec precisa de privilégios root?

Não, pode ser executado como usuário normal.

Q7: Como atualizar o BeaverSec?

Use `git pull origin main` na pasta do projeto.

Q8: Como desinstalar a instalação global?

```
sudo rm /usr/local/bin/beaver
```


12 Contribuindo

Como Adicionar um Módulo

- 1 Crie o arquivo em `beaversec/modules/`:

```
# beaversec/modules/meu_modulo.py
def run(target: str, verbose: bool = False) -> None:
    print(f"Executando contra: {target}")
```

- 2 Teste:

```
python3 main.py meu_modulo 8.8.8.8 -v
```

Como Reportar Bugs

Acesse: github.com/ojhonatanls/beaversec/issues

Como Sugerir Melhorias

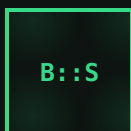
Faça um fork, crie uma branch, commit e abra um Pull Request.

Links Úteis

Repositório: github.com/ojhonatanls/beaversec

Issues: github.com/ojhonatanls/beaversec/issues

Python: python.org



*“Conhecimento compartilhado é
conhecimento multiplicado.”*

BeaverSec – Binary Enumeration & Advanced Vulnerability Exploitation Recon
© 2026 BeaverSec Team. Todos os direitos reservados.